

MediCore

A Retrieval-Augmented System for Automated ICD-10-CM Medical Coding with a Local Ollama Model

This document describes the design of a system that reads free-text clinical documentation and assigns the corresponding ICD-10-CM diagnosis codes. It covers the problem framing, the chosen architecture and why it was chosen, the trade-offs and assumptions made, measured results on the provided dataset, and a roadmap for improving the system further.

1. Problem & Framing

Medical coding is the task of translating a clinical narrative (discharge summary, encounter note, operative report) into a set of standardized codes used for billing, reporting, and analytics. The target vocabulary here is **ICD-10-CM**. The 2026 code set contains **98,186** ordered entries, of which **74,719** are valid, billable (assignable) codes; the remainder are non-billable header categories in the code hierarchy.

Three properties make this hard and shape every design decision:

- **Enormous label space.** ~75k assignable codes cannot fit in any model context window, so the system cannot simply "ask the model to pick codes". The label set must be narrowed before generation.
- **Multi-label output.** Each note maps to a variable-size set of codes (in the provided data: 1–10, mean $\approx$ 2.9). The system must decide both *which* and *how many*.
- **Specificity rules.** ICD-10-CM encodes laterality, encounter type (initial/subsequent/sequela), acuity, and combination concepts in the code itself (e.g. S52 . 502A). The correct code is often one of dozens of near-identical siblings.

The dataset provided consists of **54 labelled cases** (`icd10_cm_cases.json`) and the official CMS order file (`icd10cm_order_2026.txt`). All 136 unique gold codes are valid billable codes present in the order file, confirming the assignable set is the correct candidate space.

2. System Architecture

Because the label space is far too large for direct generation, MediCore uses a **retrieve-then-assign** (retrieval-augmented generation) pipeline. This mirrors how a human coder actually works: read the chart, abstract the codeable concepts, look them up in the index, then select the most specific supported code. The pipeline has an offline index-build stage and a four-step per-note stage.

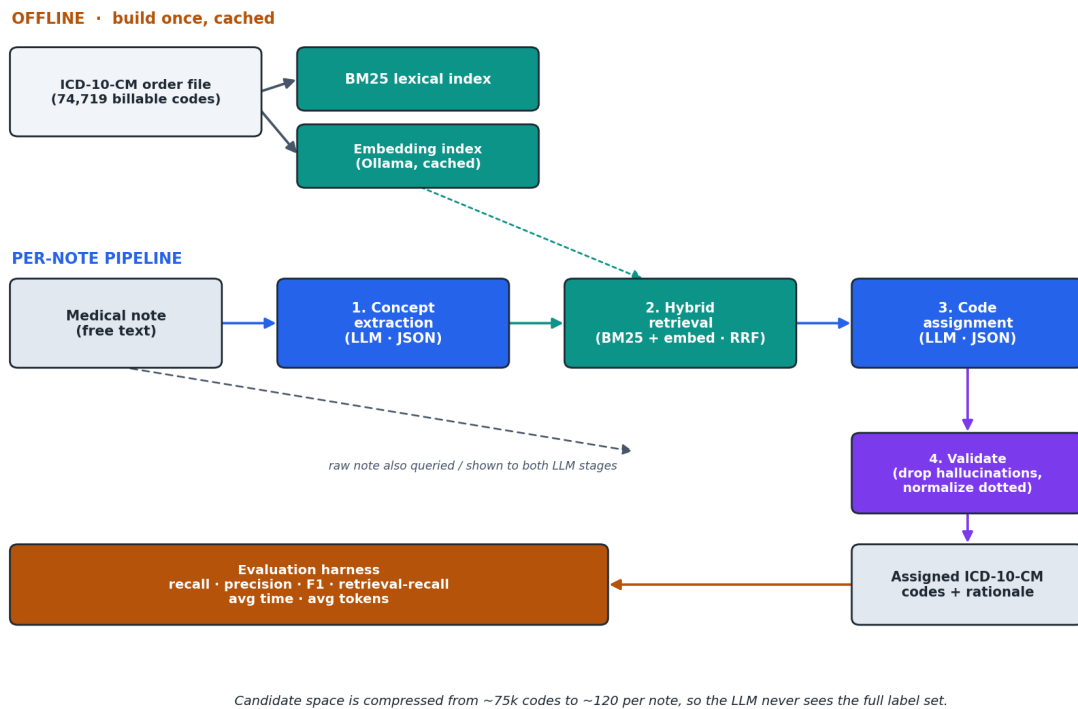


Figure 1 — MediCore end-to-end architecture. The candidate space is compressed from ~75k codes to ~120 per note before the assignment LLM is invoked.

2.1 Offline — code knowledge base

The CMS order file is parsed into the 74,719 billable codes, each with its long description. Two indexes are built over the descriptions:

- **BM25 lexical index** — a classic sparse ranker. It is fast, needs no model, and is excellent at matching exact clinical terminology (drug names, organisms, anatomical sites) that appears verbatim in descriptions.
- **Dense embedding index** — each description is embedded with an Ollama embedding model (nomic-embed-text) and cached to disk. This captures paraphrase and synonymy that BM25 misses ("heart attack" ↔ "myocardial infarction").

2.2 Per-note pipeline

- **Step 1 — Concept extraction (LLM).** The model reads the note and returns a JSON list of discrete codeable concepts (diagnoses, injuries with laterality, symptoms, procedures, external causes, status/history). This converts a long noisy narrative into precise search phrases — critical, because BM25 on the raw note is dominated by irrelevant tokens.
- **Step 2 — Hybrid retrieval.** The whole note and every extracted concept are used as queries. BM25 and embedding results are fused with **Reciprocal Rank Fusion** (RRF), a rank-based combiner that needs no score calibration, yielding a de-duplicated candidate pool of ~120 codes.
- **Step 3 — Code assignment (LLM).** The note plus the candidate list (code + description) is given to the model, which selects the codes the documentation supports, each with a short justification, as JSON. The model chooses only from the provided candidates.
- **Step 4 — Validation.** Outputs are normalized to dotted form and any code not in the candidate set / not a valid billable code is dropped, eliminating hallucinated codes.

Why concept extraction is done with an LLM rather than raw retrieval: on the provided data, BM25 over the raw note frequently fails to surface the correct code (the narrative's incidental words — "hospital", "admitted" — pull in place-of-occurrence and external-cause codes), whereas a single extracted phrase

such as "severe opioid dependence with withdrawal" retrieves the exact code (F11 . 23) at rank 1.

3. Rationale & Alternatives Considered

Alternatives weighed against the chosen RAG design:

- **Direct generation (no retrieval).** Ask the LLM to emit ICD-10 codes from parametric memory. Rejected: small local models do not reliably memorize a 75k-code vocabulary and hallucinate plausible but invalid codes; there is also no way to guarantee validity.
- **Flat multi-label classifier.** Train a 75k-way classifier. Rejected here: needs a large labelled corpus (54 cases is far too few), and the task constrains us to an Ollama model rather than a bespoke trained head.
- **Pure lexical lookup.** BM25 alone. Rejected as a full solution: it retrieves candidates well but cannot judge documentation support, specificity, or de-duplicate sibling codes — that reasoning is what the assignment LLM provides.
- **Hierarchical / two-level routing** (chapter → category → code). Viable and complementary, but adds latency and error-compounding across hops; RRF hybrid retrieval reaches the correct leaf directly.

The retrieve-then-assign design keeps the LLM responsible only for *judgement over a small candidate set* — the thing LLMs are good at — while retrieval handles *recall over the huge label space* — the thing indexes are good at. It is model-agnostic (any Ollama chat model plugs in), needs no training data, and guarantees output validity by construction.

4. Trade-offs

- **Retrieval recall is the hard ceiling.** If a gold code is never retrieved, the assignment stage cannot recover it. We therefore bias retrieval toward recall (high K, hybrid, multi-query) at the cost of a larger, noisier candidate list. The reported *retrieval recall* metric makes this ceiling explicit.
- **Two LLM calls vs one.** Concept extraction markedly improves retrieval but adds one call (latency + tokens). It can be disabled (`extract_concepts: false`) to trade accuracy for speed.
- **Candidate-list size.** More candidates → higher recall ceiling but a longer, costlier assignment prompt and more distractors for the model. 120 is a middle setting, tunable in config.
- **Embeddings vs BM25-only.** Embeddings improve paraphrase recall but require a one-time index build (~75k vectors) and extra memory. A BM25-only mode is provided for constrained environments.
- **Model size vs speed.** A small 3B model keeps latency and memory low but reasons less reliably about specificity than a larger model; the design lets you swap models without code changes.

5. Assumptions

- The assignable target space is the set of billable (flag = 1) codes in the CMS order file; non-billable headers are never assigned.
- Gold labels are treated as a complete, order-independent set per note; evaluation is set-based (no code sequencing / principal-diagnosis ordering is scored).
- Codes are compared in canonical dotted, upper-case form; the order file's undotted codes are normalized to match the gold format.
- Notes are self-contained; no external EHR context, problem list, or prior encounters are available.
- Only an Ollama model is used for all learned components (extraction, assignment, embeddings), per the task constraint.

6. Measured Results

Evaluated on all **54** provided cases with the configured Ollama models. Recall is the primary requested metric; precision/F1 are reported for context. *Micro* pools code-level hits across all notes (weights by number of codes); *macro* averages per-note scores (weights notes equally).

Metric	Micro	Macro
Recall (primary)	0.190	0.256
Precision	0.155	0.192
F1	0.170	0.210

Operational metric	Value
Cases evaluated	54
Retrieval recall (candidate ceiling)	0.754
Average time per note	23.09 s
Average tokens per note (total)	3533
· prompt tokens	3275
· completion tokens	259

Reading the numbers: the gap between *retrieval recall* and final *recall* is what the assignment LLM loses (it failed to select a retrieved gold code); the gap between retrieval recall and 1.0 is what retrieval never surfaced. This decomposition tells you where to invest next — a bigger/better assignment model vs. deeper retrieval.

Token usage is captured directly from the Ollama response fields (`prompt_eval_count`, `eval_count`); timing is wall-clock per note including retrieval. Numbers scale with the chosen model and candidate-list size, not with the design itself.

The 54-case set is small, so these figures are indicative rather than statistically tight; they establish a working baseline and a measurement harness, not a production benchmark.

7. How to Improve the System Further

- **Ontology-aware retrieval.** Exploit the ICD-10 hierarchy and the Alphabetic Index / Includes-Excludes notes to expand queries and constrain candidates to coherent sub-trees.
- **Learned reranker.** Insert a cross-encoder rerank step between retrieval and assignment to push gold codes into the top of the candidate list, raising the achievable ceiling per token.
- **Coding-guideline injection.** Provide ICD-10-CM Official Guidelines (sequencing, combination codes, 7th-character rules) as system context to improve specificity and reduce invalid combinations.
- **Constrained / grammar decoding.** Force the assignment output onto the candidate code grammar so malformed or out-of-set codes are impossible rather than filtered afterward.
- **Stronger / fine-tuned model.** A larger Ollama model, or one LoRA-tuned on coded charts, would improve the specificity judgement that dominates the remaining error.
- **Self-consistency & verification.** Sample multiple assignments and vote, or add a verifier pass that checks each code against the documentation before emitting it.
- **Human-in-the-loop.** Surface per-code justifications and confidence so a coder can accept/reject — turning the system into an assistive tool and generating new training data.

- **Scale & robustness.** Batch embedding, an ANN index (FAISS) for sub-millisecond retrieval at full scale, and evaluation on a larger, multi-institution corpus.

Appendix — reproducibility. All metrics in Section 6 are produced by `python scripts/run_eval.py`, which writes `reports/eval_summary.json` and a per-case CSV. See the README for configuration and setup.